

# Structures, Classes and Object-Oriented Foundations in C++

> **Definition to Remember**

> A str  
encaps

---

## 1. Concept Overview & Fundamentals

Structures, Classes and Object-Oriented Foundations in C++ provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

- \*\*Z

---

## 2. Technical Architecture & Memory Mechanics

### C++ Runtime Execution Architecture

```text

+-----

### Modern C++ Feature Comparison

| Modern C++ Paradigm | Legacy Equivalent | Engineering Advantage | Performance Impact |

| :--- | :

---

### 3. Core Syntax & Implementation Patterns

#### Modern C++ Template Structure

```
```cpp
// Fast I/O and standard namespace configuration
std::cout << "Executing C++ Module: Structures" << "\n";
return 0;
```
```

### 4. Real-World Applications & System Engineering

- **Quantitative Finance & HFT:** High-frequency trading platforms use modern C++ for microsecond-level execution.

---

### 5. Comprehensive Solved Problems & Code Walkthroughs

#### Problem 1: book array initializer

**Problem Description & Objective:**

**C++ Code Implementation:**

```
/*
```

```
- ===
```

```
#include <iostream>
```

```
#includ
```

```
// Use standard library namespace globally
```

```
using n
```

```
struct Book {
```

```
string
```

```
// --- Main Execution Entry Point ---
```

```
int mai
```

```
// Output formatted data to console stream
```

```
cout <
```

```
// Return zero code indicating successful program finish
```

```
return
```

```
---
```

## Problem 2: book structure

**\*\*Problem Description & Objective:\*\***

Implemen

**\*\*C++ Code Implementation:\*\***

```
```cpp
```

```
/*
```

```
- ===
```

```
#include <iostream>
```

```
#includ
```

```
// Use standard library namespace globally
```

```
using n
```

```
struct Book {
```

```
string t
```

```
void displayBook(const Book &b) {
```

```
// Outp
```

```
// --- Main Execution Entry Point ---
```

```
int main
```

```
// Output formatted data to console stream
```

```
cout <
```

```
// Return zero code indicating successful program finish
```

```
return
```

```
---
```

### Problem 3: car description function

**\*\*Problem Description & Objective:\*\***

Implemen

**\*\*C++ Code Implementation:\*\***

```
```cpp
```

```
/*
```

```
- ===
```

```
#include <iostream>
```

```
// Use standard library namespace globally
```

```
struct Car {
```

```
void printCarDescription(const Car &c) {
```

```
// --- Main Execution Entry Point ---
```

```
printCarDescription(myCar);
```

```
// Return zero code indicating successful program finish
```

```
---
```

#### **Problem 4: car structure**

**\*\*Problem Description & Objective:\*\***

**\*\*C++ Code Implementation:\*\***

```
/*
```

```
#include <iostream>
```

```
// Use standard library namespace globally
```

```
struct Car {
```

```
// --- Main Execution Entry Point ---
```

```
// Output formatted data to console stream
```

```
// Return zero code indicating successful program finish
```

```
---
```

### Problem 5: nested structure student books

**\*\*Problem Description & Objective:\*\***

**\*\*C++ Code Implementation:\*\***

```
/*
```

```
- ===
```

```
#include <iostream>
```

```
#includ
```

```
// Use standard library namespace globally
```

```
using n
```

```
struct Book {
```

```
string t
```

```
struct Student {
```

```
int id;
```

```
void displayStudentInfo(const Student &s) {
```

```
// Outp
```

```
// --- Main Execution Entry Point ---
```

```
int main
```

```
displayStudentInfo(student1);
```

```
// Return zero code indicating successful program finish
```

```
return
```

```
---
```

### **Problem 6: student modify gpa**

**\*\*Problem Description & Objective:\*\***

Implemen

**\*\*C++ Code Implementation:\*\***

```
```cpp
```

```
/*
```

```
- ===
```

```
#include <iostream>
```

```
// Use standard library namespace globally
```

```
struct Student {
```

```
void updateGPA(Student *s, double newGPA) {
```

```
// --- Main Execution Entry Point ---
```

```
// Output formatted data to console stream
```

```
updateGPA(&s1, 3.9);
```

```
// Output formatted data to console stream
```

```
// Return zero code indicating successful program finish
```

```
---
```

## 6. Exam Scoring Points & Memory Revision

> **Must-Write Points**

> **Quick Recall**